

TP 3 Couche Application Partie I

TP Réseaux Avancés ING3 IA - M1 SIC - IA

Ilyas Bambrik

Table des matières



Objectifs	3
I - Désactivation Pare-feu Windows	4
II - Exercice : Commandes FTP	6
III - Exercice : Implémentation client FTP et ftplib	8
IV - Exercice : Client TFTP-Like	11
V - Exercice : nslookup	14
VI - Exercice : Enregistrements DNS	16
VII - Exercice : Requête GET	18
VIII - Exercice : Requête HEAD	20
IX - Exercice : Entête Range	21
X - Exercice : Requête OPTIONS	23
XI - Exercice : Requête If-Modified-Since	24
XII - Exercice : Téléchargement de fichier (png, jpeg, exe) depuis le serveur Web	25
XIII - Exercice : Téléchargement de contenu avec HTTPS	27
XIV - Exercice : requests	29
XV - Exercice : Téléchargements Multiples	30
XVI - Exercice : Cookies HTTP	31

Objectifs

- Analyser les messages ARP avec Wireshark ;
- Analyser les types de messages ICMP avec Wireshark ;
- Travail à rendre : pour chaque question/étape prenez une capture d'écran complète (les captures d'écran partielles ne seront comptabilisés). Le document à consulter doit être un fichier PDF avec les réponses et captures numérotées ;
- L'organisation du document à consulté est comptabilisée dans la note ;
- Le TP sera consulté deux semaines après la publication de celui-ci ;

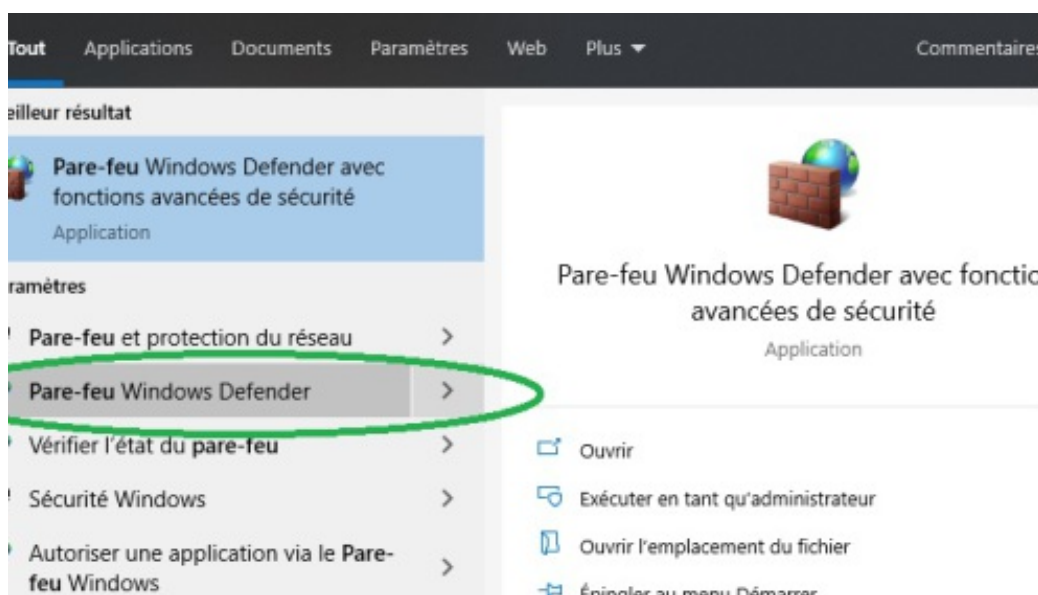
Les codes présentés dans ce support sont téléchargeables à partir du lien suivant :
https://drive.google.com/drive/folders/1-dlAgNSYj3VDPdNajqMXu4A1R7_DxgXK

Désactivation Pare-feu Windows

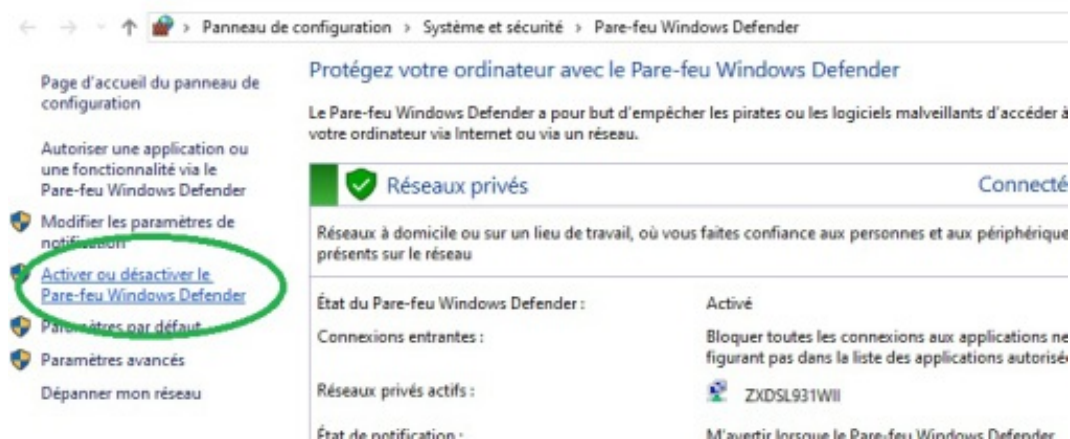
I

Avant de commencer à utiliser l'application client ftp de windows depuis le CMD, il faut désactiver le parefeu windows car l'implémentation windows fonctionne exclusivement avec le mode FTP actif.

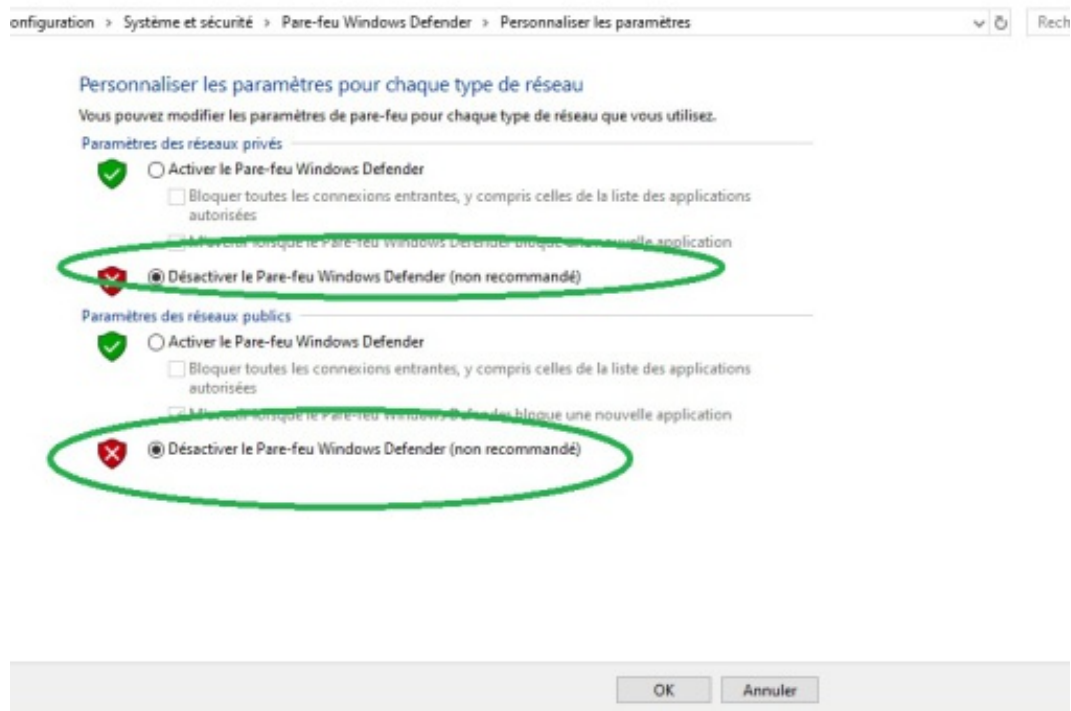
Ouvrez "Pare-feu Windows Defender" depuis le menu démarrage.



Cliquez sur activer ou désactiver le "Parefeu Windows Defender"



Désactiver le parefeu windows (après la fin des tests TP, cliquez sur "Activer le Parefeu Windows Defender")



Exercice : Commandes FTP

II

Ouvrez votre Invité de commande (CMD) et Wireshark. Capturez l'interface Wifi.

Connectez au serveur FTP 145.220.21.40 depuis votre invité de commande avec la commande suivante:

`ftp ftp.nluug.nl`

ou bien

`ftp 145.220.21.40`

Question 1

- Authentifiez vous avec un login *anonymous* (accès anonyme) et puis tapez une adresse mail avec un format valide (nom.prenom@yahoo.fr par exemple) comme mot de passe.
- Filtrez les communications sur Wireshark pour avoir seulement les échanges avec le serveur FTP (*filtrez avec adresse IP de préférence par exemple : ip.addr==145.220.21.40 afin de filtrer la communication avec le serveur FTP ftp.nluug.nl*) ;
- Quelles sont les commandes qui précèdent le nom d'utilisateur et le mot passe respectivement?

Question 2

- Exécutez la commande *ascii* après la connexion en mode anonyme. Quelle est la commande transmise par le client et le code de réponse renvoyé par le serveur ?
- Exécutez la commande *pwd*. Quelle est la commande transmise par le client et le code de réponse renvoyé par le serveur ?

Question 3

- Tapez *LS* (comme dans *Linux* pour lister le contenu du répertoire courant)
- Quelles sont les commandes FTP transmises par le client suite à l'exécution de *LS* ? (utilisez Wireshark pour la retrouver).
- Quels sont les ports TCP utilisés par le client et serveur pour transmettre le résultat (le listing du répertoire) ? Quels sont les codes transmis par le serveur suite à l'exécution de *LS* ?

Utilisez le filtre `tcp.port==21` pour repérer la connexion de contrôle et `tcp.port==20` pour la connexion de donnée (en mode actif).

Question 4

- Naviguez dans le répertoire : pour les gens qui testent ces commandes sur le serveur *145.220.21.40*,
Exécutez les commandes suivantes
CD pub/os/BSD/FreeBSD
GET README.TXT
- Repérez les commandes transmises depuis le client dans wireshark.
- Repérez le fichier téléchargé dans wireshark avec le filtre *ip.addr == 145.220.21.40 || ftp-data*.
- Exécutez la commande *binary* sur le terminale. Quelle est la commande transmise qui correspond à celle ci.

Exercice : Implémentation client FTP et ftplib

III

Lisez et testez le programme suivant (celui ci montre comment lire et écrire dans un fichier en python):

Pour tester le programme, donnez une chemin d'un à fichier source sur votre disque dur et un chemin de sortie valides. Par exemple *C:\Python27\README.txt*

```

1 # Le programme suivant permet de copier un fichier en mode binaire
2 # (exe, png, etc)
3
4 # Entrez le chemin de votre fichier a copier
5 Chemin_fichier_oreginal=input("Entrez le chemin de votre fichier :")
6 #Entrez le chemin de votre fichier :
7 Chemin_copie=input("Entrez l'emplacement de la copie :")
8 # Ouverture du fichier en mode lecture (r) en octets (b)
9 Fichier_en_lecture=open(Chemin_fichier_oreginal,"rb")
10 # utilisez un chemin de fichier de votre choix
11 Suite_doctets=[]
12 print( "#"*20)
13 for bloque_octets in Fichier_en_lecture:
14     Suite_doctets.append(bloque_octets)
15
16 # Ouverture du fichier en mode ecriture (w) en octets (b)
17 Fichier_en_ecriture=open(Chemin_copie,"wb")
18 for octets in Suite_doctets:
19     # Ecriture de la suite d'octets dans le fichier Image
20     Fichier_en_ecriture.write(octets)
21 # Fermeture du fichier
22 Fichier_en_ecriture.close()
23

```

Listing 1 - Programme pour copier un fichier binaire

Le programme précédant peut être abrégé comme suite. La méthode *read()* permet de lire le contenu entier du fichier.

```

1 Oreginal=open(input("Entrez le chemin de votre fichier :"),'rb')
2 Copie=open(input("Entrez l'emplacement de la copie :") , "wb")
3 Copie.write(Oreginal.read())
4 Copie.close()

```

Le programme python suivant permet de se connecter à un serveur FTP (port 21) et d'exécuter des commandes d'accès seulement (USER,PASS, CWD, QUIT).

- Que doit être ajouté à ce programme pour permettre l'échange de données ?


```

1 import socket
2 import re
3 ConnexionAUnServeur = socket.socket()
4
5 # Entrez l'adresse IP du serveur FTP depuis le clavier
6 print ("Entrez l'adresse IP du serveur FTP depuis le clavier :")
7 host = input()
8 port = 21 # numero de port
9 ConnexionAUnServeur.connect((host, port)) # etablisement de connexion avec le
10
11 while True:
12     recu=""
13     while True:
14         recu+=ConnexionAUnServeur.recv(2028).decode()
15         m=re.findall("\d\d\d ",recu)
16         if len(m)!=0:
17             break
18     print(recu)
19     commande=input().encode()
20     ConnexionAUnServeur.send(commande+b"\r\n")
21
22 ConnexionAUnServeur.close()
23

```

Listing 2 - Connexion à un fichier binaire

Le programme suivant utilise la librairie standard python ftplib qui permet d'exécuter n'importe quelle commande FTP. Exécutez le programme et lisez le résultat.

```

1 import ftplib
2 # ouverture d'une connexion au serveur FTP
3 connexionftp=ftplib.FTP("ftp.nluug.nl")
4 print('\n##### Fin instruction [connexionftp=ftplib.FTP("ftp.vim.org")]\n')
5 # commande login anonyme
6 connexionftp.login()
7
8 print( '\n##### Fin instruction [connexionftp.login()]\n')
9 # commande NLST
10 print( '\n'.join(connexionftp.nlst()))
11
12 print( '\n##### Fin instruction [connexionftp.nlst()]\n')
13
14 print(connexionftp.cwd("pub"))
15 print(connexionftp.cwd("ImageMagick"))
16 print('\n##### Fin instruction CD pub\ImageMagick')
17 # telechargement du fichier welcome.msg
18 connexionftp.retrbinary('RETR ImageMagick-6.9.11-29.7z', open('ImageMagick-
19 6.9.11-29.7z', 'wb').write)
20

```

Listing 3 - Ftplib

Le dernier programme montre comment exécuter des commandes système à partir d'un programme python. Ces derniers peuvent être utilisés dans un programme serveur pour implémenter des fonctionnalités similaires à FTP.

```

1 import os # bibliotheque systeme
2 import time

```

```
3 # os.mkdir cree un repertoire courant
4 # cette boucle cree les repertoires "0" jusqu'a "5"
5 for i in range(5):
6
7     os.mkdir(str(i))
8 # met le programme en pause pour 10sec
9 time.sleep(10)
10 # rmdir supprime un repertoire
11 # cette boucle supprime les repertoires "0" jusqu'a "5"
12 for i in range(5):
13     os.rmdir(str(i))
14
15 # os.listdir imprime les noms des repertoires/fichiers dans
16 # le repertoire courant
17 print( os.listdir("."))
18 # os.getcwd() est equivalente a pwd (imprime le chemin actuel)
19 print( os.getcwd())
20 # CD .. (retourne au repertoire parent)
21 os.chdir("../")
22 print( "#####",os.getcwd(),os.listdir("."))
```

Exercice : Client TFTP-Like

IV

Soit le code python du serveur du protocole TFTP-Like suivant :

```

1 # -*- coding: cp1252 (cp: codepunk) -*-
2 import socket
3 import os
4 import re
5 SocketServeur = socket.socket()
6 port = 9500
7 Chemin_racine=os.getcwd().split("\\\\")[-1]
8
9 # Ecoute de connexion sur toutes les interfaces (0.0.0.0)
10 SocketServeur.bind(("0.0.0.0", port))
11
12 print("Lancement serveur")
13 SocketServeur.listen(10)
14 while True:
15     # Attente d'une connexion (accept)
16     ConnexionAUnClient, addrclient = SocketServeur.accept()
17
18
19     print( "Connexion de la machine = ", addrclient)
20     while True:
21         # Format des commandes transmis par le client : "Type_Commande:NomFichier"
22         # Type_Commande == 3 changer le repertoire courant (Change Working
23         Dirrectory)
24         # Type_Commande == 2 demander la liste des fichiers dans le repertoire
25         courant
26         # Type_Commande == 1 ( par exemple si la commande recue == "1:File.png"
27         signifie telecharger "File.png" )
28         # Type_Commande == 0 Si la commande commence par "0:" ce la signifie que
29         le client souhaite deconnecter
30         # par exemple "0:", "0:CYA" et "0:leaving" signifient deconnexion
31         try:
32             Commande=ConnexionAUnClient.recv(1024)
33             # decouper la commande par le caractere ":"
34             Arguments=Commande.decode("utf-8").split(":")
35             # le premier argument represente le type de la commande
36             Taille_commande=int(Arguments[0])
37             Nom_Fichier=Arguments[1]
38             if(Taille_commande==0):# commande de deconnexion
39                 ConnexionAUnClient.send(b"200 k, bye")
40                 # L'envoi d'une sequence "--\r\n\r\n" delimitte la fin d'une
41                 reponse
42                 ConnexionAUnClient.send(b"--\r\n\r\n")
43                 # Sortir de la boucle de traitement while
44                 break

```

```

40         elif(Taille_commande==1):# commande de telechargement de
fichier
41             # Nom_Fichier ne doit pas contenir ".." ou / (tentative de
parcourir les repertoires )
42             if( ".." in Nom_Fichier or "/" in Nom_Fichier ):
43                 ConnexionAUnClient.send(b"501 DIRECTORY TRAVERSAL DENIED")
44                 ConnexionAUnClient.send(b"--\r\n\r\n")
45                 continue # Esquiver le reste des instructions
46             try:
47                 # Lire le fichier designé par la commande utilisateur
48                 Fichier_Upload=open(Nom_Fichier,"rb")
49                 for octets in Fichier_Upload:
50                     # Transmettre chaque sequence d'octets lu vers le
client
51                     ConnexionAUnClient.send(octets)
52                     # Fermer le fichier lu
53                     Fichier_Upload.close()
54                     ConnexionAUnClient.send(b"--\r\n\r\n")
55             except:
56                 ConnexionAUnClient.send(b"500 Fichier introuvable")
57                 ConnexionAUnClient.send(b"--\r\n\r\n")
58         elif(Taille_commande==2):
59             # os.listdir(".") renvoie la liste des fichier / repertoires dans
le repertoire courant (comme ls et dir)
60             Liste_Des_Fichiers=os.listdir(".")
61             ConnexionAUnClient.send("\n".join(Liste_Des_Fichiers).encode("utf-
8"))
62             ConnexionAUnClient.send(b"--\r\n\r\n")
63         elif(Taille_commande==3):
64             try:
65                 os.chdir(Nom_Fichier)
66                 if( Chemin_racine in os.listdir(".") ):
67                     os.chdir(Chemin_racine)
68                     ConnexionAUnClient.send(b"501 Nope (srsly!)")
69             else:
70                 ConnexionAUnClient.send(("200 OK "+Nom_Fichier).encode(
"utf-8"))
71             except:
72                 ConnexionAUnClient.send(("404 Le repertoire "+Nom_Fichier+"
est introuvable").encode("utf-8"))
73                 ConnexionAUnClient.send(b"--\r\n\r\n")
74         except:
75             try:
76                 # Fermeture de la connexion avec le client
77                 ConnexionAUnClient.close()
78             except:
79                 pass
80             break
81     print ("Deconnexion de :",addrclient)
82
83
84 SocketServeur.close()
85

```

Question 1

Écrivez le code d'un client permettant la lecture / envoi de commande TFTP_Like et la récupération du résultat. Le programme client doit gérer l'occurrence des erreurs avec la clause *try - except* ainsi que permettre l'exécution des quatre commandes.

Indice :

Voir l'exemple suivant pour le format des commande et réponse de TFTPLike :



```

Python 2.7.9 Shell
File Edit Shell Debug Options Windows Help
Type "copyright", "credits" or "license()" i
>>> ===== RESTART
>>>
2:
code.py
Help-20181222T115202Z-001.zip
Licences Informatiques.rar
PyQt5-5.5-gpl-Py3.4-Qt5.5.0-x64.exe
Rep
TFTP_LIKEpy2.py
TFTP_LIKEpy3.py
wordnet.zip
wrar561fr.exe
--

3:Rep
200 OK Rep
--

1:run.py
import os

import __future__
print ( "Hello",os.environ["USERNAME"])
--

0:babay
200 k, bye
--

Exiting
>>>

```

Question 2

Changez le code du serveur afin de le rendre capable de traiter les requêtes de plusieurs clients simultanément (voir le premier TP sur le multi-threading).

Exercice : nslookup

V

Ouvrez votre invité de commande et tapez `ipconfig /all` (ifconfig pour linux) et ensuite repérez votre configuration DNS pour l'interface avec la quelle vous êtes connecté sur Internet. Vous pouvez voir l'adresse IP de votre serveur DNS (voir la figure en bas)

```

Windows PowerShell
Carte réseau sans fil Connexion au réseau local* 3 :

Statut du média. . . . . : Média déconnecté
Suffixe DNS propre à la connexion. . . :
Description. . . . . : Microsoft Wi-Fi Direct Virtual Adapter #2
Adresse physique . . . . . : 4A-51-B7-D4-C7-1D
DHCP activé. . . . . : Oui
Configuration automatique activée. . . : Oui

Carte réseau sans fil Wi-Fi :

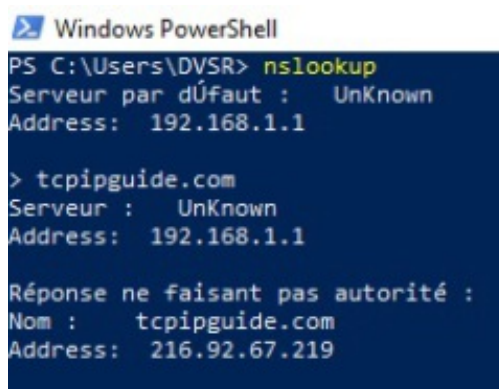
Suffixe DNS propre à la connexion. . . :
Description. . . . . : Intel(R) Dual Band Wireless-AC 7260
Adresse physique . . . . . : 48-51-B7-D4-C7-1D
DHCP activé. . . . . : Oui
Configuration automatique activée. . . : Oui
Adresse IPv6. . . . . : fd00:664b:6dc1:aa00:b152:4f6a:ae26:9875(préféré)
Adresse IPv6 temporaire . . . . . : fd00:664b:6dc1:aa00:f520:ce2c:5415:d0b1(préféré)
Adresse IPv6 de liaison locale. . . . : fe80::b152:4f6a:ae26:9875%18(préféré)
Adresse IPv4. . . . . : 192.168.1.6(préféré)
Masque de sous-réseau. . . . . : 255.255.255.0
Bail obtenu. . . . . : mercredi 12 décembre 2018 18:16:01
Bail expirant. . . . . : vendredi 14 décembre 2018 16:51:13
Passerelle par défaut. . . . . : 192.168.1.1
Serveur DHCP . . . . . : 192.168.1.1
IAID DHCPv6 . . . . . : 105402807
DUID de client DHCPv6. . . . . : 00-01-00-01-23-18-30-F1-B8-6B-23-9A-92-8E
Serveurs DNS. . . . . : 192.168.1.1
  
```

- Tapez `nslookup` (outil de résolution de nom de domaine) et ouvrez *wireshark* avec votre interface correspondante.
- Écrivez le nom de domaine `tcpipguide.com` dans l'invité de commande `nslookup` ;
- Appliquez le filtre suivant dans Wireshark : `dns.qry.name == "tcpipguide.com"`
- L'envoi de la requête DNS retourne l'adresse IP du nom de domaine "`tcpipguide.com`". La requête transmise par le Resolver (client `nslookup`) est une demande d'enregistrement A == Adresse record (voir la capture d'écran Wireshark). Ce type d'enregistrement fait la correspondance entre un nom de domaine et l'adresse IP correspondante (voir le cours).
- Par la suite la réponse correspondante à la requête est l'enregistrement de type A contenant l'adresse IP de "`tcpipguide.com`".

Question

- Tapez `nslookup` (outil de résolution de nom de domaine) et ouvrez *wireshark* (ainsi que votre interface correspondante) .
- Écrivez le nom de domaine `tcpipguide.com` dans l'invité de commande `nslookup` ;
- Appliquez le filtre suivant dans Wireshark : `dns.qry.name == "tcpipguide.com"`

- L'envoi de la requête DNS retourne l'adresse IP du nom de domaine "*tcpipguide.com*". La requête transmise par le Resolver (client nslookup) est une demande d'enregistrement A == *Adresse record* (voir la capture d'écran Wireshark). Ce type d'enregistrement fait la correspondance entre un nom de domaine et l'adresse IP correspondante (voir le cours).
- Par la suite, la réponse correspondante à la requête est l'enregistrement de type A contenant l'adresse IP de "*tcpipguide.com*".



```
Windows PowerShell
PS C:\Users\DVR> nslookup
Serveur par défaut : UnKnown
Address: 192.168.1.1

> tcpipguide.com
Serveur : UnKnown
Address: 192.168.1.1

Réponse ne faisant pas autorité :
Nom : tcpipguide.com
Address: 216.92.67.219
```

Exercice : Enregistrements DNS

VI

Question 1

- Pour récupérer l'adresse IPv6 d'un nom de domaine (si ce domaine possède une adresse IPv6 public), tapez dans l'invité de nslookup les commandes suivantes :

```
set type=AAAA
```

```
google.com
```

- Les deux commandes précédentes permettent de spécifier que le type d'enregistrement demandé est l'adresse IPv6 public de google.com. *set type=AAAA* change le type d'enregistrement demandé par la requête DNS.

Si vous procédez à exécuter un *ping* vers *www.lequotidien-oran.com* et *ns002.naltis.com*, vous remarquerez que les deux machines auront la même adresse IP (*voir le déroulement du ping, l'adresse IP des deux domaines est identique*).

```
Invite de commandes

C:\Users\DVSR>ping www.lequotidien-oran.com

Envoi d'une requête 'ping' sur ns002.naltis.com [91.121.146.205] avec 32 octets de données :
Réponse de 91.121.146.205 : octets=32 temps=133 ms TTL=48
Réponse de 91.121.146.205 : octets=32 temps=75 ms TTL=48
Réponse de 91.121.146.205 : octets=32 temps=73 ms TTL=48
Réponse de 91.121.146.205 : octets=32 temps=83 ms TTL=48

Statistiques Ping pour 91.121.146.205:
    Paquets : envoyés = 4, reçus = 4, perdus = 0 (perte 0%),
    Durée approximative des boucles en millisecondes :
        Minimum = 73ms, Maximum = 133ms, Moyenne = 91ms

C:\Users\DVSR>ping ns002.naltis.com

Envoi d'une requête 'ping' sur ns002.naltis.com [91.121.146.205] avec 32 octets de données :
Réponse de 91.121.146.205 : octets=32 temps=74 ms TTL=48
Réponse de 91.121.146.205 : octets=32 temps=74 ms TTL=48
Réponse de 91.121.146.205 : octets=32 temps=74 ms TTL=48
Réponse de 91.121.146.205 : octets=32 temps=74 ms TTL=48

Statistiques Ping pour 91.121.146.205:
```

Question 2

- Pour récupérer l'enregistrement CNAME, alias d'un nom de domaine si celui-ci possède un alias, tapez dans l'invité de *nslookup* les commandes suivantes :

```
set type=CNAME
```

```
www.lequotidien-oran.com
```


- Les deux commandes précédentes permettent d'avoir l'alias de *www.lequotidien-oran.com*. Dans cet exemple, *www.lequotidien-oran.com* pointe sur *ns002.naltis.com* (voir la capture d'écran).
- *ns002.naltis.com* possède l'adresse IP 91.121.146.205
- CNAME == *Canonical Name* ou bien nom canonique.

Question 3

Rechercher le nom de domaine correspondant à l'adresse IP 145.220.21.40 avec le type d'enregistrement PTR :

set type=PTR

145.220.21.40

```
> set type=PTR
> 145.220.21.40
Server: UnKnown
Address: 192.168.1.1

Non-authoritative answer:
40.21.220.145.in-addr.arpa      name = ftp.nluug.nl
> |
```

Question 4

- Pour récupérer les noms des serveurs mail dans un domaine, il suffit de changer le type de l'enregistrement demandé (set type=MX) :

set type=MX

google.com

- Les noms des serveurs retournés sont des noms de domaine SMTP. La valeur de préférence permet de choisir entre plusieurs serveurs mail (une valeur de préférence plus petite indique une préférence supérieur par rapport aux autres serveurs mail disponibles). Dans cet exemple : Le serveur mail préféré de google est == *asmpx.l.google.com* (la valeur de préférence minimale);
- Plusieurs serveurs mail peuvent coexister dans un domaine pour que le système soit tolérant au pannes (s un serveur ne peut pas recevoir le mail entrant, le prochain serveur mail selon la valeur de préférence est choisi pour l'échange du mail en question) ;
- MX= *Mail eXchanger* ;

Exercice : Requête GET

VII

Le code suivant connecte au serveur HTTP *www.tcipguide.com* et envoie une simple requête GET (si vous essayez d'accéder à un autre serveur web, utilisez le nom du serveur web correspondant et remplacez *www.tcipguide.com* dans le code et la requête HTTP). La requête dans cet exemple demande la *page d'accueil* de ce serveur web (/):

```

1 # -*- coding: cp1252 -*-
2 # -*- @author: codek -*-
3
4 import socket
5 ConnexionAuServeurHTTP=socket.socket()
6
7 # nom de domaine du serveur web
8 Serveur="www.tcipguide.com"
9 # numero de port
10 Port=80
11
12 # connecte au serveur distant
13 ConnexionAuServeurHTTP.connect((Serveur,Port))
14 # transmet la requete GET. Les headers de la requetes se terminent pas CRLF
15 # (\r\n)
16 # la requete se termine par CRLFCRLF (\r\n\r\n)
17 ConnexionAuServeurHTTP.send(b"GET / HTTP/1.1\r\nHost:www.tcipguide.
18 com\r\nConnexion:close\r\n\r\n")
19
20 # Les deux entetes HTTP dans cette requete sont :
21 # 1) [Host:www.tcipguide.com] (nomdu domaine du serveur web)
22 # 2) [Connexion:close] (pour cloturer la connection apres l'envoi de la reponse)
23
24 # recevoir le contenu de la page ( lire le contenu reçu du buffer en morceaux
25 [chaîne de caractere] de taille maximale <= 1024 )
26
27 rec=ConnexionAuServeurHTTP.recv(1024)
28 while True:
29     # apres la cloture de la connexion, le nombre d'octets lu sera d'une taille
30     de 0
31     if len(rec)==0:
32         break
33     # imprime le contenu du morceau reçu
34
35     print(rec.decode("unicode_escape"),)
36     # lire un nouveau morceau
37     rec=ConnexionAuServeurHTTP.recv(1024)

```

Le résultat de la requête *GET /* sera la réponse suivante (entêtes de la réponse serveur web + la page web renvoyée par le serveur) :

```

Python 2.7.9 Shell
File Edit Shell Debug Options Windows Help
Python 2.7.9 (default, Dec 10 2014, 12:24:55) [MSC v.1500 32 bit (Intel)] on win32
Type "copyright", "credits" or "license()" for more information.
>>> ===== RESTART =====
>>>
HTTP/1.1 200 OK
Date: Thu, 13 Dec 2018 18:39:20 GMT
Server: Apache/2.4.37
Accept-Ranges: bytes
Content-Length: 20757
Content-Type: text/html

<html>

<head>
<meta http-equiv="Content-Language" content="en-us">
<meta name="GENERATOR" content="Microsoft FrontPage 6.0">
<meta name="ProgId" content="FrontPage.Editor.Document">
<meta http-equiv="Content-Type" content="text/html; charset=windows-1252">
<title>Welcome to The TCP/IP Guide!</title>
<script language="JavaScript" fptype="dynamicanimation">
<!--
function dynAnimation() {}
function clickSwapImg() {}
//-->

```

Réponse Serveur avec code 200
et taille de la ressource

CRLF CRLF pour marquer le début de la ressource transmise par le serveur

Exercice : Requête HEAD

VIII

Le code suivant connecte au serveur HTTP *www.tcpipguide.com* et envoie une simple requête *HEAD* pour obtenir des informations sur une ressource demandée (comme dans le premier exemple, il s'agit de la page d'accueil).

```

1 # -*- @author: codek -*-
2
3 ## Lire l'explication de DemoHTTP1_GET avant de lire ce document
4
5 # requete HEAD (obtention des informations sur le contenu d'une ressource)
6
7 import socket
8 ConnexionAuServeurHTTP=socket.socket()
9 Serveur="www.tcpipguide.com"
10 Port=80
11 ConnexionAuServeurHTTP.connect((Serveur,Port))
12
13 # l'url de cette requete = / (racine)
14 ConnexionAuServeurHTTP.send(b"HEAD / HTTP/1.1\r\nHost:www.tcpipguide.
    com\r\nConnexion:close\r\n\r\n")
15 rec=ConnexionAuServeurHTTP.recv(1024)
16 # lire le contenu jusqu'a la fin
17 while True:
18     if len(rec)==0:
19         break
20     print(rec.decode("unicode_escape"),)
21     rec=ConnexionAuServeurHTTP.recv(1024)
22

```

Le résultat de la requête sera la réponse suivante (entêtes descriptifs de la ressource sans la ressource elle même : taille de la ressource 20757 octets, type de la ressource texte html, le serveur accepte les réponses partielles) :

```

>>>
HTTP/1.1 200 OK
Date: Thu, 13 Dec 2018 18:59:06 GMT
Server: Apache/2.4.37
Accept-Ranges: bytes
Content-Length: 20757
Content-Type: text/html
>>> |

```

Réponse serveur

Exercice : Entête Range

IX

Le code suivant connecte au serveur HTTP *www.tcpipguide.com* et envoie une requête GET avec l'entête *Range* pour obtenir une partie de la ressource (les octets dans les positions entre 500-1000).

```
1 # -*- @author: codek -*-
2
3 ## Lire l'explication de DemoHTTP1_GET avant de lire ce document
4
5 # Range (demande de contenu partiel) / [ reponse serveur ContentRange - Code 206
  contenu partiel]
6
7 import socket
8 ConnexionAuServeurHTTP=socket.socket()
9 Serveur="www.tcpipguide.com"
10 Port=80
11 ConnexionAuServeurHTTP.connect((Serveur,Port))
12 # L'entete [Range: bytes=500-1000] indique que le client souhaite recevoir
   seules les octets de 500 jusqu'a 1000
13 # Demande des caracteres dans l'intervalle 500-1000
14 ConnexionAuServeurHTTP.send(b"GET / HTTP/1.1\r\nHost:www.tcpipguide.
   com\r\nRange: bytes=500-1000\r\nConnexion:close\r\n\r\n")
15 rec=ConnexionAuServeurHTTP.recv(1024)
16 while True:
17     if len(rec)==0:
18         break
19     print(rec.decode("unicode_escape"),)
20     rec=ConnexionAuServeurHTTP.recv(1024)
21
```

Le résultat de la requête sera la réponse suivante (les octets de 500 à 1000 de la page demandée) :

Exercice : Requête OPTIONS

X

Le code suivant connecte au serveur HTTP *www.tcpipguide.com* et envoie une requête OPTIONS pour obtenir les méthodes (GET, PUT, HEAD, DELETE, etc) supportées pour la ressource désignée.

```

1 # -*- coding: cp1252 -*-
2 # OPTIONS :
3
4 import socket
5 ConnexionAuServeurHTTP=socket.socket()
6 Serveur="www.tcpipguide.com"
7 Port=80
8 ConnexionAuServeurHTTP.connect((Serveur,Port))
9 ConnexionAuServeurHTTP.send(b"OPTIONS / HTTP/1.1\r\nHost:www.tcpipguide.
  com\r\nConnexion:close\r\n\r\n")
10
11 rec=ConnexionAuServeurHTTP.recv(1024)
12 while True:
13     if len(rec)==0:
14         break
15     print(rec.decode("unicode_escape"),)
16     rec=ConnexionAuServeurHTTP.recv(1024)
17

```

Le résultat de la requête sera la réponse suivante (les méthodes permises sur l'url donné):

```

>>> ===== RESTART =====
>>>
HTTP/1.1 200 OK
Date: Thu, 13 Dec 2018 19:18:40 GMT
Server: Apache/2.4.37
Allow: POST,OPTIONS,HEAD,GET  Methodes supportées
Content-Length: 0
Content-Type: text/html
>>> |

```

Exercice : Requête If-Modified-Since

XI

Le code suivant connecte au serveur HTTP *www.tcpipguide.com* et envoie une simple requête GET pour obtenir la ressource si la ressource n'a pas été modifiée ultérieurement à la date *If-Modified-Since*.

```

1 # -*- @author: codek -*-
2
3 #Last-Modified: Mon, 07 Jun 2004 00:26:52 GMT Code reponse 304
4 import socket
5 ConnexionAuServeurHTTP=socket.socket ()
6 Serveur="www.tcpipguide.com"
7 Port=80
8 ConnexionAuServeurHTTP.connect ((Serveur,Port))
9 # requete avec header If-Modified-Since (telecharger la ressource si celle-ci a
   ete modifi[&e] au dela de la data specifi[&e] par If-Modified-Since
10 ConnexionAuServeurHTTP.send(b"GET /free/diagrams/tcpswunack.png HTTP/1.1\r\nHost:
   www.tcpipguide.com\r\nIf-Modified-Since: Mon, 07 Jun 2005 00:26:52 GMT\r\nConnexion:
   close\r\n\r\n")
11
12 rec=ConnexionAuServeurHTTP.recv(1024)
13 while True:
14     if len(rec)==0:
15         break
16     print(rec.decode("unicode_escape"),)
17     rec=ConnexionAuServeurHTTP.recv(1024)
18

```

Le résultat de la requête sera la réponse suivante (le serveur a répondu avec le code 304 pour dire que la ressource n'a pas été modifiée Mon, 07 Jun 2004 00:26:52 GMT):



```

>>> ----- RESTART -----
>>>
HTTP/1.1 304 Not Modified
Date: Thu, 13 Dec 2018 19:27:36 GMT
Server: Apache/2.4.37
ETag: "57de-3dc3a54612b00"

```

Le serveur répond avec un code que la ressource n'a pas été modifiée

Après l'exécution du code précédant, le serveur ne transmet pas la ressource car celle-ci n'a pas été modifiée ultérieurement à la date indiquée par l'entête If-Modified-Since (Mon, 07 Jun 2005 00:26:52 GMT).

Question

Modifier la valeur de l'entête If-Modified-Since , afin que le serveur revoie la ressource demandée.

Exercice : Téléchargement de fichier (png, jpeg, exe) depuis le serveur Web

Le téléchargement de fichiers se passe exactement de la même façon que la demande de contenu HTML avec une requête GET. Cependant, pour enregistrer le contenu du fichier demandé, on doit d'abord séparer les entêtes de la réponse serveur du contenu. Pour ceci, il suffit de lire jusqu'à la séquence CR LF CR LF (`\r\n\r\n`) pour retrouver le premier octet de la ressource (ligne 16). En suite, au lieu d'afficher avec *print* les octets reçus, il suffit de d'enregistrer ceux dans un fichier.

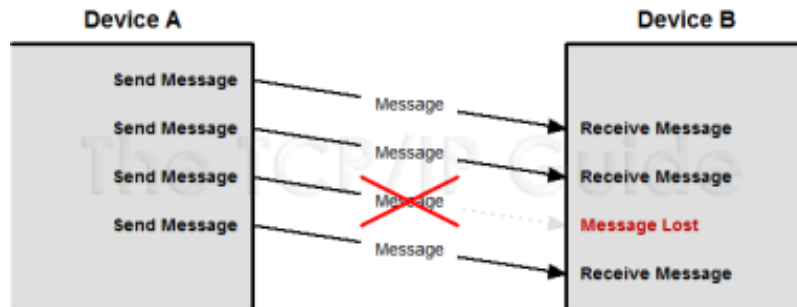
Le code suivant envoie une requête pour le fichier `/free/diagrams/tcpswunack.png` (ligne 9) et l'enregistre dans un fichier `tcpswunack.png`. Après la réception du fichier complet, celui-ci est fermé avec la méthode `close()` (ligne 29).

Après l'exécution du code, un fichier `tcpswunack.png` sera créé dans le même répertoire que le code source. Ouvrez celui pour voir le résultat.

```
1 # -*- @author: codek -*-
2
3 import socket
4 ConnexionAuServeurHTTP=socket.socket()
5 Serveur="www.tcpipguide.com"
6 Port=80
7 ConnexionAuServeurHTTP.connect((Serveur,Port))
8 # demande le fichier /free/diagrams/tcpswunack.png avec la methode GET
9 ConnexionAuServeurHTTP.send(b"GET /free/diagrams/tcpswunack.png HTTP/1.1\r\nHost:
  www.tcpipguide.com\r\nConnection:close\r\n\r\n")
10
11 rec=ConnexionAuServeurHTTP.recv(1024)
12 # rec est une chaine de caractere reçu
13 # rec contiendra les entetes HTTP de la reponse serveur ainsi que la ressource
14 # la sequence b'\r\n\r\n' separe les entetes HTTP du contenu de la ressource
15 # ainsi, pour lire les caracteres de la ressources seulement, il suffit de lire
  a partir du caractere qui suit la sequence "\r\n\r\n"
16 rec=rec[rec.find(b"\r\n\r\n")+4::]
17
18 # ouvrir un fichier en mode ecriture en octets ("wb" == write, bytes)
19 fichier=open("tcpswunack.png","wb")
20
21 while True:
22     if len(rec)==0:
23         break
24     # au lieu d'imprimer le contnu reçu sur ecran
```

```
25 fichier.write(rec)
26 rec=ConnexionAuServeurHTTP.recv(1024)
27
28
29 fichier.close()
30
```

L'image contenue dans "tcpswunack.png" est la suivante :



Exercice : Téléchargement de contenu avec HTTPS

XIII

Question 1

La différence entre HTTP et HTTPS est le fait que dans HTTPS, tout juste après l'établissement de connexion TCP entre le client et le serveur, les deux parties s'échangent leurs certificats numériques contenant la clé publique (*RSA*). En suite, une clé privée est partagée (*AES*) entre le client et serveur.

Après la distribution des clés, les requêtes client et les réponses serveur sont les même que HTTP mais sont chiffrées. L'échange de certificat et les opérations de chiffrement sont assurées par le module *SSL* (*Secure Socket Layer*) en python (*ligne 6 et ligne 12*). Le code suivant connecte au serveur HTTPS coursera.com, et télécharge le fichier pdf contenu de l'url */api/certificate.v1/pdf/HDC2GWUR3YT4* dans un fichier *"fichier.pdf"*.

```

1
2 # -*- coding: cp1252 -*-
3 # -*- @author: codek -*-
4 import socket
5 connexionTCP=socket.socket()
6 import ssl
7 Serveur="www.coursera.org"
8 Port=443
9
10 connexionTCP.connect((Serveur,Port))
11 ConnexionAuServeurHTTPS=ssl.create_default_context().wrap_socket(connexionTCP,
    server_hostname=Serveur)
12
13 ConnexionAuServeurHTTPS.send(b"GET /api/certificate.v1/pdf/HDC2GWUR3YT4 HTTP/1.
    1\r\nHost:www.coursera.org\r\nConnection:close\r\n\r\n")
14 rec=b""
15 while 1:
16     if b"\r\n\r\n" in rec:
17         break
18     rec+=ConnexionAuServeurHTTPS.recv(1024)
19 rec=rec[rec.find(b"\r\n\r\n")+4::]
20 fichier_=open("fichier3.pdf","wb")
21 while True:
22     if len(rec)==0:
23         break
24     fichier_.write(rec)
25     rec=ConnexionAuServeurHTTPS.recv(1024)
26 fichier_.close()
27 ConnexionAuServeurHTTPS.close()
28

```

Question 2

Utilisez la commande ping pour trouver l'adresse IP de la machine www.datacamp.com.

Question 3

Ouvrez Wireshark et filtrez pour trouver les paquet IP contenant des segments TCP, transmis ou reçus de www.datacamp.com. Ensuite, utilisez votre navigateur pour accéder à la ressource <https://www.datacamp.com>.

Repérez les échanges TLS entre le client et le serveur après le Three Way Handshake.

Exercice : requests

XIV

Question

Le programme suivant montre comment utiliser la bibliothèque requests au lieu des sockets pour interroger un serveur Web (équivalent au 1er programme de la série). Ce module doit être installé (*pip install requests*) :

Lisez et testez le programme suivant :

```
1 import requests# librairie requests
2 url="http://www.tcpipguide.com/"
3
4 reponse=requests.get(url)
5 print(reponse.content)
```

Exercice : Téléchargements Multiples

XV

Question

Écrivez une fonction *telecharger_ressources* :

- Cette fonction prend en paramètre : a) un socket (connexion au serveur) *con_serveur*, b) le nom ou adresse IP du serveur *server_name*, c) liste d'URL de ressources à télécharger *ressources*, d) liste des noms des fichiers où chaque ressource doit être enregistrée *chemins*;
- Cette fonction doit utiliser une seule connexion TCP représentée par *con_serveur* afin de télécharger toutes les ressources dans la liste *ressources* et enregistrer chaque résultat dans le fichier correspondant de la liste *chemins*.
- Après l'envoi d'une requête, vous devez recevoir la réponse jusqu'à la fin avant de transmettre la requête pour la ressource suivante.

Voir ci-dessous comment la fonction *telecharger_ressources* doit être appelée avec l'exemple d'URL (la liste d'URLs *ressources* = ["/free/diagrams/osinotation.png", "/free/diagrams/tcpswunack.png"], *chemins* = ["test.png", "test2.png"]):

```
1 import socket
2 ConnexionAuServeurHTTP=socket.socket ()
3 Serveur="www.tcpipguide.com"
4 Port=80
5 ConnexionAuServeurHTTP.connect ((Serveur,Port))
6
7 def telecharger_ressources (con_serveur,server_name,ressources,chemins):
8     # à implémenter
9     pass
10 telecharger_ressources(ConnexionAuServeurHTTP, Serveur, ["/free/diagrams
    /osinotation.png", "/free/diagrams/tcpswunack.png"], ["test.png", "test2.png"])
```

Donnez un programme qui utilise cette fonction pour télécharger des ressource à partir d'un serveur HTTP.

Indice :

- Pour maintenir la connexion avec le serveur vous devez utiliser la version *HTTP 1.1* dans votre requête et l'entête général *Connection: keep-alive*.
- Dans la réponse serveur, recherchez l'entête *Content-Length* pour retrouver la taille de la ressource à recevoir

Exercice : Cookies HTTP

XVI

Le code serveur HTTP suivant vous permet de saisir une chaîne de caractères et la placer dans la variable `vous_cookie` (Ligne 6). La valeur de celle-ci sera interpolée dans la chaîne de caractères contenue dans la variable `header` (`Set-Cookie:%s` Ligne 7).

```
1 import socket
2 s=socket.socket()
3 s.bind(("0.0.0.0",80))
4 s.listen(30)
5 contenu="<h1> Page HTML</h1>"
6 vous_cookie=input("Entrez la valeur de la variable vous_cookie: ")
7 header='HTTP/1.1 200 OK\r\nConnexion:close\r\nDate: Thu, 17 Jan 2019 13:18:42
  GMT\r\nServer: Apache/2.4.37\r\nAccept-Ranges: bytes\r\nContent-Length: %
  d\r\nContent-Type: text/html\r\nSet-Cookie:%s\r\n\r\n'%(len(contenu),vous_cookie
  )
8 while True:
9     con,_=s.accept()
10    con.send(header.encode())
11    con.send(contenu.encode())
12    con.close()
```

Question

- Lancez Wireshark sur l'interface *localhost* et filtrez afin de retrouver seulement les requêtes *GET* reçues sur le port *TCP 80* (`http.request.method==GET`).
- Exécutez le code serveur précédant et connectez avec un navigateur web à ce serveur (`http://localhost`). Avant que le serveur se lance, vous devez introduire la valeur de la variable *vous_cookie* au clavier.
- Repérez la chaîne de caractères que vous avez introduit au clavier dans l'entête *Cookie* : de la requête de votre navigateur avec Wireshark (prenez une capture d'écran de l'entête *Cookie*).